

Guide d'installation : Environnement de travail ML

À lire avant le premier TP. Ce guide t'accompagne pas à pas pour installer un environnement Python moderne sur Mac, Linux ou Windows (WSL). À la fin, tu pourras lancer n'importe quel TP du cours avec une seule commande.

Table des matières

- 1. [Vue d'ensemble](#)
- 2. [Mac](#)
- 3. [Linux \(Ubuntu/Debian\)](#)
- 4. [Windows : WSL2](#)
- 5. [Installation commune \(uv + Python 3.13\)](#)
- 6. [VSCode](#)
- 7. [Vérification finale](#)
- 8. [Lancer un TP](#)
- 9. [Alternatives sans VSCode](#)

1. Vue d'ensemble

L'environnement repose sur trois outils :

Outil	Rôle	Pourquoi
uv	Gestionnaire Python + venvs	10-100× plus rapide que pip, remplace pyenv + pip + venv
Python 3.13	Interpréteur	Version installée et gérée par uv
VSCode	Éditeur + Jupyter	Interface la plus complète pour les TPs

Le Makefile de chaque TP utilise uv pour créer automatiquement l'environnement virtuel.

2. Mac

2.1 Homebrew (si pas déjà installé)

Homebrew est le gestionnaire de paquets de référence sur Mac.

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/inst
```

Suivre les instructions affichées, notamment l'ajout de Homebrew au PATH (indiqué en fin d'installation).

Vérification :

```
brew --version  
# → Homebrew 4.x.x
```

2.2 Outils de base

```
# Git (contrôle de version, nécessaire pour uv et les TPs)  
brew install git  
  
# make (interpréteur de Makefile : souvent déjà présent sur Mac)  
brew install make
```

Vérification :

```
git --version    # → git version 2.x.x  
make --version   # → GNU Make 4.x
```

Apple Silicon (M1/M2/M3) : Homebrew s'installe dans /opt/homebrew/. Si des commandes ne sont pas trouvées après installation, ajouter à ~/.zshrc :

```
export PATH="/opt/homebrew/bin:$PATH"
```

Puis relancer le terminal ou `source ~/.zshrc` .

3. Linux (Ubuntu/Debian)

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl build-essential make
```

Vérification :

```
git --version
make --version
curl --version
```

4. Windows : WSL2

WSL2 (Windows Subsystem for Linux) permet d'utiliser un vrai terminal Linux directement sur Windows. C'est la seule façon d'utiliser les Makefiles et `uv` dans de bonnes conditions sur Windows.

4.1 Activer WSL2

Ouvrir **PowerShell en tant qu'administrateur** et exécuter :

```
wsl --install
```

Cette commande installe WSL2 avec Ubuntu par défaut. Redémarrer l'ordinateur quand demandé.

Si `wsl --install` ne fonctionne pas (Windows < 21H2) :

```
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /n
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
# Redémarrer, puis :
wsl --set-default-version 2
```

Puis télécharger Ubuntu depuis le Microsoft Store.

4.2 Première configuration Ubuntu

Au premier lancement, Ubuntu demande un nom d'utilisateur et un mot de passe (choisir des valeurs simples, elles servent pour `sudo`).

```
# Mettre à jour les paquets
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl build-essential make
```

4.3 Accès aux fichiers

- Depuis WSL, tes fichiers Windows sont accessibles dans `/mnt/c/Users/<ton-nom>/`
- Depuis Windows, les fichiers WSL sont accessibles via `\\wsl$\\Ubuntu\\home\\<ton-nom>`
- **Recommandé** : travailler directement dans le home Linux (`~/`) pour de meilleures performances

Vérification WSL :

```
wsl --list --verbose
# → Ubuntu    Running  2    (VERSION doit être 2)
```

5. Installation commune (uv + Python 3.13)

Cette section s'applique sur Mac, Linux **et** dans le terminal WSL.

5.1 Installer uv

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Relancer le terminal, ou sourcer le profil :

```
source ~/.bashrc    # Linux / WSL
source ~/.zshrc     # Mac (zsh par défaut)
```

Vérification :

```
uv --version  
# → uv 0.x.x
```

5.2 Installer Python 3.13 avec uv

```
uv python install 3.13
```

uv télécharge et installe Python 3.13 dans `~/.local/share/uv/python/` : pas besoin de sudo, pas de conflit avec le Python système.

Vérification :

```
uv python list  
# → cpython-3.13.x-... ✓ (installé)  
  
uv run --python 3.13 python --version  
# → Python 3.13.x
```

5.3 Adapter les Makefiles au projet

Les Makefiles des TP utilisent la variable `PYTHON`. Pour les faire pointer sur Python 3.13 géré par uv, on crée un alias global :

```
# Ajouter dans ~/.bashrc (Linux/WSL) ou ~/.zshrc (Mac)  
alias python3.13='uv run --python 3.13 python'
```

Ou bien passer la variable directement à chaque `make` :

```
PYTHON="uv run --python 3.13 python" make install
```

Alternative plus propre : exporter la variable pour toute la session :

```
export PYTHON="uv run --python 3.13 python"  
make install
```

6. VSCode

6.1 Installation

OS	Méthode
Mac	<code>brew install --cask visual-studio-code</code> ou code.visualstudio.com
Linux	<code>sudo snap install code --classic</code> ou code.visualstudio.com
Windows	Télécharger depuis code.visualstudio.com : s'intègre automatiquement avec WSL

Vérification :

```
code --version  
# → 1.x.x
```

6.2 Extensions indispensables

Installer depuis le terminal (marche sur toutes les plateformes) :

```
# Python et Jupyter : obligatoires  
code --install-extension ms-python.python  
code --install-extension ms-toolsai.jupyter  
code --install-extension ms-toolsai.jupyter-keymap  
  
# Confort de développement : recommandées  
code --install-extension ms-python.black-formatter  
code --install-extension charliermarsh.ruff  
code --install-extension streetsidesoftware.code-spell-checker  
code --install-extension eamodio.gitlens
```

Ou depuis l'interface VSCode : Ctrl+Shift+X (ou Cmd+Shift+X sur Mac), rechercher par nom.

6.3 Configuration WSL + VSCode

Sur Windows, ouvrir VSCode depuis le terminal WSL :

```
# Dans le terminal WSL, se placer dans le dossier du TP
cd ~/cours_ml/01_non_supervise
code .
```

VSCode installe automatiquement un serveur WSL et s'y connecte. En bas à gauche de VSCode, tu vois `WSL: Ubuntu` quand c'est actif.

6.4 Sélectionner le bon kernel Jupyter dans VSCode

1. Ouvrir un notebook `.ipynb`
2. En haut à droite, cliquer sur le kernel actuel (ex. `Python 3.x.x`)
3. Choisir `Select Another Kernel` → `Python Environments`
4. Sélectionner le venv du TP (ex. `.tpnonsup/bin/python`) : il apparaît après `make install`

Si le venv n'apparaît pas : `Ctrl+Shift+P` → `Jupyter: Refresh Kernel` puis réessayer.

7. Vérification finale

Exécuter ce script dans le terminal pour vérifier que tout est en place :

```
python3 << 'EOF'
import sys

checks = [
    ("Python >= 3.13", sys.version_info >= (3, 13)),
]

# Vérification des modules standards
for mod in ["venv", "pip"]:
    try:
        __import__(mod)
        checks.append((f"module {mod}", True))
    except ImportError:
        checks.append((f"module {mod}", False))

print("=== Vérification de l'environnement ===\n")
all_ok = True
for label, ok in checks:
    icon = "✓" if ok else "x"
    print(f" [{icon}] {label}")
    if not ok:
        all_ok = False

print()
print("🟢 Tout est prêt !" if all_ok else "⚠️ Certains points nécessitent attention.")
EOF
```

Et pour uv :

```
echo "---- uv ----"
uv --version

echo "---- Python 3.13 via uv ----"
uv run --python 3.13 python --version

echo "---- make ----"
make --version | head -1
```

8. Lancer un TP

Une fois l'environnement installé, lancer un TP se fait en 3 commandes :

```
# 1. Se placer dans le dossier du TP
cd ~/cours_ml/01_non_supervise

# 2. Installer les dépendances (première fois uniquement)
PYTHON="uv run --python 3.13 python" make install

# 3. Lancer Jupyter
make notebook    # Jupyter Notebook classique
# ou
make lab         # JupyterLab (interface plus moderne)
```

`make help` dans n'importe quel dossier TP affiche toutes les commandes disponibles.

9. Alternatives sans VSCode

Si tu ne peux pas ou ne veux pas utiliser VSCode, voici les alternatives classées par facilité d'accès :

Option A : Google Colab (aucune installation)

colab.research.google.com : accès via compte Google.

Avantages :

- Rien à installer, tourne dans le navigateur
- GPU/TPU gratuits disponibles
- Idéal pour le TP Deep Learning (torch, transformers)

Démarrage avec les notebooks du cours :

1. Dans Colab : Fichier → Importer un notebook → Charger depuis l'ordinateur
2. Uploader le `.ipynb` du TP
3. Uploader les CSV si besoin (icône dossier à gauche → Importer)
4. Les packages manquants s'installent avec `!pip install <package>` dans une cellule

Limitation : l'environnement est réinitialisé à chaque session (variables perdues, packages à réinstaller).

Option B : JupyterLab en local

Si tu as installé uv et Python 3.13 mais pas VSCode, tu peux utiliser JupyterLab directement dans le navigateur.

```
cd ~/cours_ml/01_non_supervise
PYTHON="uv run --python 3.13 python" make install
make lab
```

JupyterLab s'ouvre à `http://localhost:8888` : interface complète avec explorateur de fichiers, terminal intégré, notebooks.

Option C : Kaggle Notebooks (aucune installation)

kaggle.com/code : accès via compte Kaggle (gratuit).

Avantages :

- GPU gratuit (30h/semaine)
- Beaucoup de packages pré-installés (sklearn, torch, xgboost, transformers)
- Bonne alternative à Colab

Démarrage : New Notebook → importer le `.ipynb` via File → Import Notebook .

Option D : GitHub Codespaces

github.com/codespaces : nécessite un compte GitHub.

Avantages :

- Environnement VSCode complet dans le navigateur
- Persistant entre les sessions (contrairement à Colab)

- 60h/mois gratuites

Démarrage : créer un repo GitHub, y déposer les TPs, puis Code → Codespaces → New codespace .

Option E : PyCharm

jetbrains.com/pycharm : version Community gratuite.

Supporte les notebooks Jupyter nativement. Configurer l'interpréteur sur le venv créé par `make install` .

Tableau comparatif

	VSCode (recommandé)	Google Colab	JupyterLab	Kaggle	Codespaces
Installation	Locale	Aucune	Locale	Aucune	Aucune
GPU	Non (CPU)	Gratuit limité	Non (CPU)	30h/sem	Non
Persistance	Oui	Non	Oui	Oui	Oui (60h/mois)
Makefiles	Oui	Non	Terminal intégré	Non	Oui
Hors ligne	Oui	Non	Oui	Non	Non
Notebooks IPYNB	Oui	Oui	Oui	Oui	Oui

Récapitulatif des commandes clés

```
# Installer uv
curl -Lsf https://astral.sh/uv/install.sh | sh

# Installer Python 3.13
uv python install 3.13

# Vérifier Python 3.13
uv run --python 3.13 python --version

# Installer les dépendances d'un TP
cd cours_ml/<dossier_tp>
PYTHON="uv run --python 3.13 python" make install

# Lancer Jupyter
make notebook # ou make lab

# Lancer Jupyter sans Makefile (directement)
uv run --python 3.13 jupyter notebook

# Voir les commandes disponibles d'un TP
make help

# Nettoyer l'environnement
make clean
```